

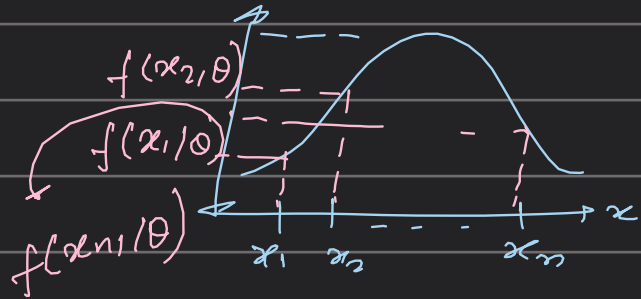
SLAM-Checkpoint-3

- Maximum Likelihood
- &
- Individual compatibility

Data Association - ML + IC

↳ the goal is to find the optimal way to fit a distribution to the data.

↓
eg: here we want to find the optimal way to fit x_i in this bell curve



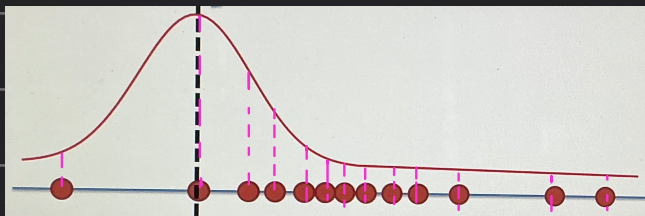
$$f(x_1, \theta) \cdot f(x_2, \theta) \cdot \dots \cdot f(x_n, \theta)$$

↓
Likelihood of x_i for given parameter(s) θ

$$L(\theta | x_{1:n}) = \prod_{i=1}^n f(x_i | \theta)$$

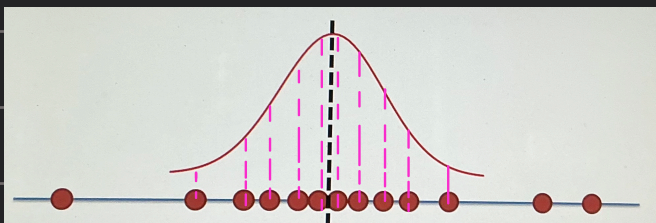
Annotations:
- Likelihood: points to $L(\theta | x_{1:n})$
- parameter: points to θ (eg: μ, σ)
- observed values: points to $x_{1:n}$
- model's density fcn: points to $f(x_i | \theta)$
- parameter: points to θ
- independent variable: points to x_i

eg: mean as parameter



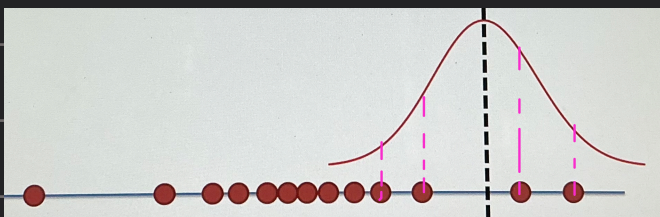
↳ the product of pdfs seems low so the likelihood is low

↳ away from mean



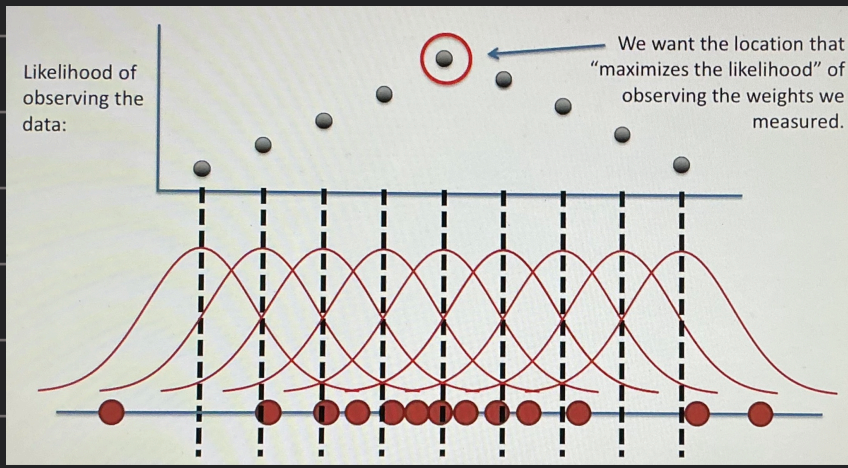
↳ the product of pdfs is high

↳ so better chance of maximizing the likelihood near mean



↳ the product of pdfs seems low so the likelihood is low

↳ away from mean



Q: here the parameter
lost which we are finding
the likelihood of z_i
in the normal distribution
is mean

$$\text{Likelihood} = \frac{1}{(2\pi\sqrt{|S|})} e^{-d^2/2}$$

d : Mahalanobis distance

$$d = \underbrace{(g_m - r)^T S^{-1} (g_m - r)}_{\text{matrix}}$$

$S = \text{cov} + R$ → sensor measurement noise covariance
↓
uncertainty in cone's pose estimation

```

214  def MaxLikelihoodAndIndividualComp(perceived_cones, real_cones, cov=None, R=None, gate_threshold=9.21):
215      if cov is None:
216          cov = np.eye(2) * 0.5
217      if R is None:
218          R = np.eye(2) * (NOISE_STD ** 2)
219
220      S = cov + R
221      S_inv = np.linalg.inv(S)
222      det_S = np.linalg.det(S)
223
224      matches = []
225
226      for p in perceived_cones:
227          best_index = -1
228          best_lik = -np.inf
229
230          for j, r in enumerate(real_cones):
231              innovation = np.array([p[0] - r[0], p[1] - r[1]])
232              d2 = innovation @ S_inv @ innovation.T
233              if d2 > gate_threshold: # individual compatibility gate
234                  continue
235
236              lik = np.exp(-0.5 * d2) / (2.0 * np.pi * np.sqrt(det_S))
237
238              if lik > best_lik:
239                  best_lik = lik
240                  best_index = j
241
242      matches.append(best_index)
243      return np.array(matches)

```